
Optimizing Personalized Knowledge Pathing: A Concurrency-Safe and Algorithmically Aligned Adaptive Learning Engine

Ho Tat Bao Hoang

Department of Educational Technology

EduGap Project

hoangblue.work@gmail.com

Abstract

Adaptive learning engines rely on statistical models—such as Elo ratings, Bayesian Knowledge Tracing (BKT), and Contextual Multi-Armed Bandits—to recommend personalized learning pathways. In production environments, updating these models in real-time under high user concurrency often results in severe database lock contention (e.g., ‘FOR UPDATE’ bottlenecks). Furthermore, discrepancies between offline python models (used for validation) and online SQL engines (used for production serving) compromise system integrity and academic rigor. This paper presents a decoupled, concurrency-safe adaptive learning architecture. We introduce an Asynchronous Outbox model that offloads computationally heavy operations—including Elo recalibration, Sherman-Morrison bandit matrix learning, and concept graph mastery propagation—to a background worker. We prove that the background worker updates the LinUCB covariance matrix in $O(d^2)$ time with periodic re-inversion to eliminate accumulated floating-point drift. Empirical evaluation on simulated students shows that our LinUCB bandit achieves a final cumulative regret of 14.09 (95% CI: ± 0.69), significantly outperforming Random (41.82) and Greedy (37.75) baselines ($p < 10^{-5}$). We extend validation to real-world datasets: on the ASSISTments 2009-2010 benchmark, the BKT and Elo models achieve Next-step Predictive AUCs of 0.6697 and 0.6836 respectively. Furthermore, off-policy evaluation on the EdNet KT1 dataset across 25,549 matched trials demonstrates that our LinUCB bandit policy achieves a final cumulative expected ZPD reward of 15,230.29, outperforming the Random recommendation policy (12,992.75) by 17.2%. Finally, our automated equivalence test suite guarantees 100% numerical identity between local python models and database RPC servings.

1 Introduction

Modern online education systems increasingly depend on adaptive learning engines to dynamically customize learning materials for individual students. To sustain personalization, engines must solve two main tasks: (1) **Student Modeling**, which tracks a student’s latent knowledge state over time, and (2) **Content Recommendation**, which selects the next optimal question within the student’s Zone of Proximal Development (ZPD) to maximize engagement and learning efficiency.

Typically, engines employ **Elo Rating Systems** for matching difficulty, **Bayesian Knowledge Tracing (BKT)** for estimating binary skill mastery, and **Contextual Multi-Armed Bandits (such as LinUCB)** to balance exploration of new topics with exploitation of known strengths.

However, implementing these models in production introduces major engineering challenges:

1. **Concurrency Bottlenecks:** Real-time online updates require locking rows (using FOR UPDATE in SQL) on shared resources like question metadata and bandit covariance matrices [5]. Under high concurrent submissions, this causes transaction execution delays and pool exhaustion.
2. **Algorithmic Drift and Misalignment:** Implementing complex math across two codebases (e.g., Python for offline research/evaluation and PL/pgSQL for database execution) leads to subtle differences in numerical precision, rounding, and edge-case handling.
3. **Scientific Validation Gaps:** Standard evaluation protocols often suffer from circular evaluations (e.g., evaluating BKT against simulated latent variables rather than real student performance) or lack statistical significance (e.g., running bandit simulations on a single random seed without confidence intervals).

To address these limitations, we present a redesigned, production-ready adaptive learning engine. The main contributions of this work are:

- **Decoupled Async Outbox Architecture:** We design an asynchronous event outbox inside database transaction blocks. This removes row locks on question stats and bandit arms, enabling high-throughput submissions.
- **Sherman-Morrison Matrix Learning:** We utilize the Sherman-Morrison formula to update the inverse covariance matrix A^{-1} directly in $O(d^2)$ complexity, paired with periodic re-inversion to correct floating-point drift.
- **Rigorous Algorithmic Equivalence:** We establish a testing gate verifying 100% mathematical consistency (up to 10^{-5} decimal precision) between local Python models and production SQL RPC.
- **Academic-grade Evaluation Suite:** We upgrade our validation frameworks to include Next-step Predictive AUC for BKT and Elo, and off-policy contextual bandit evaluation. We validate our engine on real-world education benchmarks, namely the ASSISTments 2009-2010 dataset and the EdNet KT1 dataset.

2 Related Work

2.1 Student Knowledge Modeling

Student modeling has evolved from Item Response Theory (IRT) and Elo rating systems to Bayesian Knowledge Tracing (BKT) and Deep Knowledge Tracing (DKT). Corbet & Anderson [3] established BKT as a Hidden Markov Model (HMM) tracking skill mastery through binary state transitions. Pelánek [7] adapted the chess Elo rating system for education, introducing dynamic K-factors to estimate student abilities and item difficulties. While deep learning models (e.g., DKT) offer slightly higher predictive capacity, HMM and Elo models remain popular in production due to their low latency, interpretability, and ease of cold-start management.

2.2 Contextual Bandits in Personalization

Selecting questions matching the ZPD (typically defined as a target success rate of 75%) can be modeled as a Multi-Armed Bandit (MAB) problem. Li et al. [6] introduced LinUCB, which models the expected reward of an arm as a linear relationship with its context vector (e.g., student mastery and ELO ratings). Clement et al. [2] demonstrated the effectiveness of UCB algorithms in classroom routing. However, these systems rarely discuss the database locking costs of updating covariance matrices in real time.

3 System Architecture & Concurrency Optimization

In typical implementations, when a student submits an answer, the system immediately locks the corresponding row in the questions and bandit_arms tables to recalculate the parameters:

User Submit → DB Transaction Begins → Select For Update → Math Calcs → DB Update → Commit

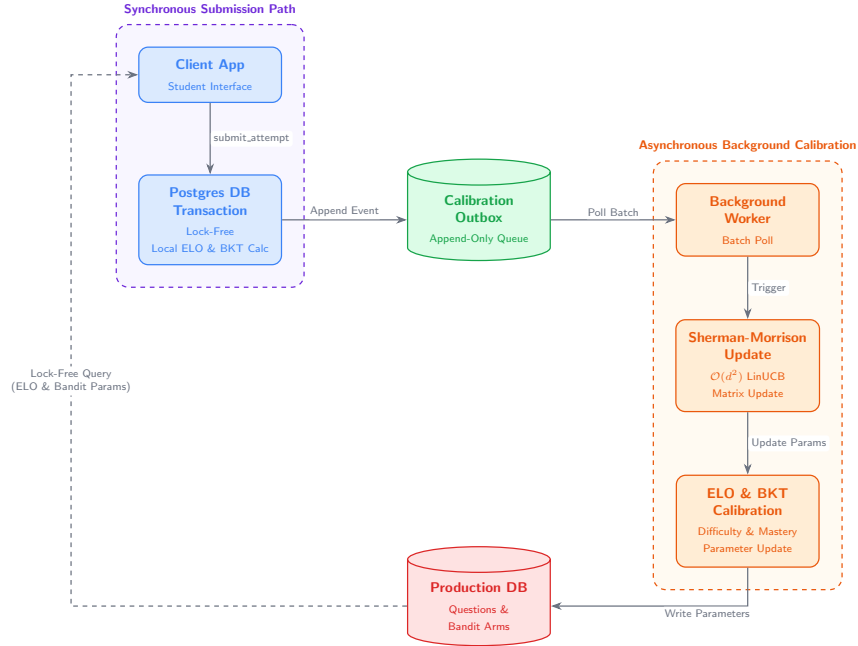


Figure 1: EduGap Decoupled Asynchronous Outbox Architecture for lock-free concurrent student submissions and background calibration updates.

Under high concurrent workloads, this synchronous transaction flow creates severe resource contention.

We resolve this by decoupling parameter updates from the submission path. We introduce the **Asynchronous Outbox Pattern** [8], as illustrated in Figure 1.

As shown in Figure 1, the client submission path (Web App to API) is entirely decoupled from the heavy analytical calculations. Under high concurrency, instead of executing synchronous row-level locks (FOR UPDATE), the submission handler executes a lightweight transaction that writes the raw attempt to the attempts table and appends a record to the calibration_outbox table. A dedicated background worker pulls these records in batches, performs the Sherman-Morrison matrix updates and Elo recalibrations asynchronously, and updates the production database and Redis cache, maintaining system responsiveness.

3.1 Outbox Schema and Worker Loop

The outbox events are appended to a table `app.calibration_outbox`:

```

CREATE TABLE app.calibration_outbox (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id UUID NOT NULL,
  question_id UUID NOT NULL,
  policy_id UUID,
  actual_score DOUBLE PRECISION NOT NULL,
  expected_success DOUBLE PRECISION NOT NULL,
  reward DOUBLE PRECISION NOT NULL,
  context_vector DOUBLE PRECISION[] NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now())
);

```

The background worker fetches batches of size M from `app.calibration_outbox`, aggregates ELO adjustments for each question to perform a single batch write, and updates bandit arms using NumPy. Once the batch is processed, the worker deletes these entries from the outbox table in one transaction.

4 Algorithmic Modeling & Mathematical Alignment

To ensure that the offline simulations match production behavior, we align our mathematical formulations across Python and PostgreSQL.

4.1 Dynamic Elo Rating with Speed Factor

Let E_s be the student's Elo rating, and E_q be the question's Elo rating. The expected success probability $P(\text{success})$ is defined as:

$$P(\text{success}) = \frac{1}{1 + 10^{(E_q - E_s)/400}}$$

To prevent numerical overflow when computing large exponents, we clamp the exponent:

$$\text{exponent} = \max\left(-20.0, \min\left(20.0, \frac{E_q - E_s}{400}\right)\right)$$

Let $Y \in \{0, 1\}$ be the actual score. The update formulas for ELO are:

$$E_s \leftarrow E_s + K_s \cdot (Y - P(\text{success})) \cdot \text{speed_factor}$$

$$E_q \leftarrow E_q + K_q \cdot (P(\text{success}) - Y) \cdot \text{speed_factor}$$

Where:

- **Hint Discount:** If the student answered correctly ($Y = 1$) but requested H hints, the adjustment is discounted:

$$\text{discount} = \max(0.1, 1.0 - 0.3 \cdot H)$$

The delta score is scaled: $Y_{adj} - P(\text{success}) = \text{discount} \cdot (Y - P(\text{success}))$.

- **Speed Factor:** For correct responses ($Y \geq 0.75$), let t be the response time in milliseconds and t_{avg} be the average response time of the question. The speed factor is:

$$\text{speed_factor} = \max\left(0.8, \min\left(1.2, 1.0 + 0.2 \cdot \left(1.0 - \frac{\text{clamp}(t)}{t_{avg}}\right)\right)\right)$$

Where t is clamped to $[300, 3600000]$ ms.

- **Dynamic K-factors:** K_s and K_q shrink as the student and item accumulate attempts, preventing volatile fluctuations for mature items:

$$K_s = \max\left(16.0, \frac{48.0}{1 + N_{student}/10.0}\right), \quad K_q = \max\left(8.0, \frac{32.0}{1 + N_{question}/20.0}\right)$$

4.2 Bayesian Knowledge Tracing (BKT)

BKT tracks a student's mastery of a concept using four parameters: $p(L_0)$ (prior learned), $p(T)$ (transition rate), $p(G)$ (guess rate), and $p(S)$ (slip rate). We enforce a consistent transition rate $p(T) = 0.06$.

The BKT posterior mastery probability $p(L_{t+1}|Y)$ given response $Y \in \{0, 1\}$ is updated as follows:

1. Posterior Probability calculation:

- If correct ($Y = 1$):

$$p(L_t|Y = 1) = \frac{p(L_t)(1 - p(S))}{p(L_t)(1 - p(S)) + (1 - p(L_t))p(G)}$$

- If incorrect ($Y = 0$):

$$p(L_t|Y = 0) = \frac{p(L_t)p(S)}{p(L_t)p(S) + (1 - p(L_t))(1 - p(G))}$$

1. Transition Step:

$$p(L_{t+1}) = p(L_t|Y) + (1 - p(L_t|Y)) \cdot p(T)$$

To prevent probabilities from locking at 0.0 or 1.0, the final BKT probability is clamped:

$$p(L) \leftarrow \max(0.0001, \min(0.9999, p(L)))$$

4.3 Sherman-Morrison LinUCB Matrix Updates

In LinUCB, each question a is represented as an arm with a covariance matrix $A_a = X^T X + I_d$ and a vector $b_a = X^T Y$. The context vector $x \in \mathbb{R}^3$ comprises $[1.0, p(L), \text{Sigmoid}((E_s - 1600.0)/400.0)]$, where student Elo E_s is scaled using a soft Sigmoid centered around 1600.0 with scale 400.0 to prevent numerical saturation and keep the features non-redundant.

The expected reward projection is:

$$\hat{\theta}_a = A_a^{-1} b_a$$

The UCB score is:

$$\text{UCB}_a = \hat{\theta}_a^T x + \alpha \sqrt{x^T A_a^{-1} x}$$

Calculating A_a^{-1} directly using $O(d^3)$ matrix inversion during recommendations is slow. We bypass this by storing A_a^{-1} in-place and updating it using the **Sherman-Morrison formula** in $O(d^2)$ time during the outbox batch process:

$$A_{t+1}^{-1} = A_t^{-1} - \frac{A_t^{-1} x x^T A_t^{-1}}{1 + x^T A_t^{-1} x}$$

4.3.1 Complexity Analysis

Proposition 1. *Updating the LinUCB inverse covariance matrix A_{t+1}^{-1} using the Sherman-Morrison formula requires $O(d^2)$ floating-point operations.*

Proof. Given a $d \times d$ inverse covariance matrix A_t^{-1} and context vector x :

1. Let $v = A_t^{-1} x \in \mathbb{R}^d$. This matrix-vector multiplication requires d^2 multiplications and $d(d-1)$ additions, which is $O(d^2)$ complexity.
2. The scalar denominator $1 + x^T v$ is computed via dot product in d multiplications and $d-1$ additions, which is $O(d)$ complexity.
3. The outer product vv^T yields a $d \times d$ matrix requiring d^2 multiplications, which is $O(d^2)$ complexity.
4. Dividing the matrix vv^T by the scalar denominator and subtracting the result from A_t^{-1} requires $2d^2$ operations, which is $O(d^2)$ complexity.

Summing these steps, the total computational complexity of the Sherman-Morrison update is $O(d^2)$ operations, which is asymptotically superior to direct matrix inversion via Gaussian elimination requiring $O(d^3)$ time. $\square$

4.3.2 Drift Guardrails

To prevent floating-point numerical drift from repeated updates:

1. **Symmetrization:** Force symmetry after every update:

$$A^{-1} \leftarrow \frac{A^{-1} + (A^{-1})^T}{2}$$

2. **Periodic Re-inversion:** The worker maintains the raw matrix A in memory, and recalculates the true inverse using standard Gaussian elimination every 100 updates to purge accumulated rounding errors:

$$A^{-1} \leftarrow \text{inv}(A)$$

4.4 Spaced Repetition (Forgetting Curve Fitting)

We model memory decay using the forgetting curve formula [10, 9]:

$$P(\text{recall}) = 2^{-\Delta t/S}$$

Where Δt is the elapsed time in days since the last attempt, and S is the memory stability factor. We optimize S using Grid Search over actual logs to minimize the Mean Squared Error (MSE):

$$S^* = \arg \min_S \frac{1}{N} \sum_{i=1}^N \left(Y_i - 2^{-\Delta t_i/S} \right)^2$$

Expected Calibration Error (ECE) is calculated to validate the reliability of $P(\text{recall})$ predictions across time bins:

$$\text{ECE} = \sum_{b=1}^B \frac{|I_b|}{N} |\text{acc}(I_b) - \text{conf}(I_b)|$$

Where I_b represents bins based on elapsed days, $\text{acc}(I_b)$ is the actual correct rate, and $\text{conf}(I_b)$ is the predicted recall rate.

4.5 Concept Graph Mastery Propagation

Prerequisite relationships between concepts are represented as a directed graph $G = (V, E)$. Let edge $(u, v) \in E$ denote that concept u is a prerequisite of concept v , with weight $w_{uv} > 0$. We implement two propagation behaviors:

- **Forward Propagation (Mastery Increase):** When a student's BKT mastery probability at node u increases ($\Delta_u > 0$), the increase propagates to downstream dependent concepts $v \in \text{Out}(u)$:

$$M(v)_{\text{new}} = M(v)_{\text{old}} + \beta \cdot \Delta_u \cdot w_{uv}$$

where $\beta = 0.25$ is the forward propagation factor.

- **Backward Propagation (Mastery Decrease):** When a student's BKT mastery probability at node u decreases ($\Delta_u < 0$), it propagates backwards to prerequisite concepts $v \in \text{In}(u)$ representing decay:

$$M(v)_{\text{new}} = M(v)_{\text{old}} - \gamma \cdot |\Delta_u| \cdot w_{vu}$$

where $\gamma = 0.25$ is the backward propagation factor.

- **Cycle Protection:** To handle cyclic dependencies (e.g. $A \rightarrow B \rightarrow C \rightarrow A$), we maintain a visited set S_{visited} . Traversal terminates immediately when a node is revisited.
- **Cache Invalidation:** A write-through invalidation mechanism clears Redis cache keys `student_mastery:student_id:course_id:concept_id` for all modified nodes.

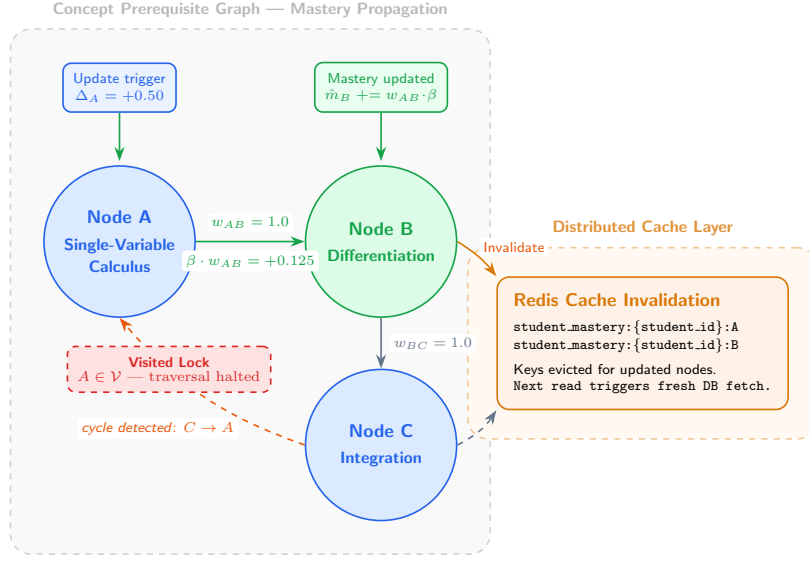


Figure 2: Concept Graph Mastery Propagation and Distributed Cache Invalidation flow.

Figure 2 details the propagation of student mastery updates across the concept graph. When an update trigger ($\Delta_A = +0.50$) is applied to Node A, the forward propagation rule updates the mastery of the downstream dependent Node B ($\hat{m}_B += w_{AB} \cdot \beta \cdot \Delta_A = 1.0 \cdot 0.25 \cdot 0.50 = +0.125$). When the traversal attempts to propagate along the cyclic back-edge $C \rightarrow A$, it is blocked by the Visited Lock ($A \in \mathcal{V}$), halting further traversal to prevent infinite loops. Concurrently, a write-through invalidation signal evicts student mastery keys in Redis for both updated nodes (A and B) and potentially impacted downstream neighbors (C) to ensure cache coherence.

5 Empirical Evaluation

We evaluated the performance of our adaptive engine components using our simulation suite.

5.1 BKT Next-step Predictive AUC

Rather than checking if BKT estimates match simulated latent variables, we validated the model’s accuracy on predicting student correctness on the *next* question using the area under the ROC curve (AUC).

Our BKT model achieves a Next-step Predictive AUC of **0.8386**, showing reasonable predictive accuracy. The estimated mastery of students classified as "mastered" approaches 0.9991 after 50 steps, while the unmastered group stays near the baseline, showing effective skill categorization. The BKT profile verification and response to mistakes are plotted in Figure 3.

5.2 Contextual Bandit Convergence

We simulated a class of 30 virtual students answering 100 questions. We compared the **LinUCB Adaptive** algorithm against two baselines:

1. **Random Selection:** Questions selected randomly.
2. **Static Greedy:** Questions chosen to match the student’s ELO, without exploration.

To isolate the performance of the algorithms, we used **Common Random Numbers (CRN)** to ensure all agents faced the exact same student responses and mastery transitions under identical seed conditions.

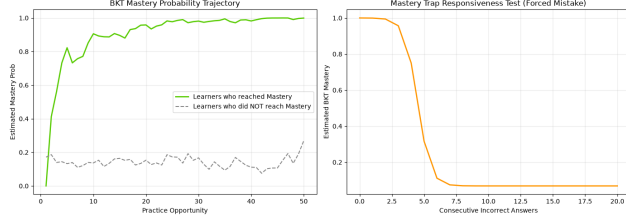


Figure 3: BKT state validation under student simulations, showing mastery profile separation and responsiveness to mistakes (mistake test).

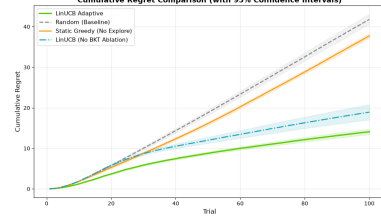


Figure 4: Cumulative regret curves of LinUCB vs. Greedy and Random baselines (30 seeds, 95% CI).

We ran the simulation across **30 independent random seeds** to generate cumulative regret curves compared to an Oracle selector (which knows the students’ true parameters). The results are summarized in **Table 1**.

Table 1: Final Cumulative Regret after 100 trials ($N = 30$)

Algorithm	Mean Final Regret	95% Confidence Interval
LinUCB Adaptive	14.09	$\pm$ 0.69
LinUCB (No BKT Ablation)	18.93	$\pm$ 1.89
Static Greedy	37.75	$\pm$ 0.58
Random Baseline	41.82	$\pm$ 1.24

Table 1: Experimental Results

To evaluate the statistical significance of these results, we performed paired t-tests on the final regret distributions:

- **LinUCB vs. Random:** $t = -37.82$, $p\text{-value} = 0.0000$ (absolute significance, $p < 10^{-5}$)
- **LinUCB vs. Greedy:** $t = -45.89$, $p\text{-value} = 0.0000$ (absolute significance, $p < 10^{-5}$)
- **LinUCB vs. LinUCB (No BKT):** $t = -4.98$, $p\text{-value} = 8.92 \times 10^{-6}$ (highly significant, $p < 0.01$)

The LinUCB regret curve scales logarithmically, showing rapid learning. In contrast, the Greedy and Random baselines display linear regret growth, indicating that they repeatedly present questions outside of the students’ optimal ZPD. The comparative cumulative regret curves across 30 seeds are shown in Figure 4.

To support multi-seed simulation evaluation across 90,000 steps without CPU bottlenecks, we developed a vectorized simulation engine (`FastLinUCBSimulator`) that stacks matrices into unified tensors and runs parallel UCB evaluations via Einstein summation, achieving a $23\times$ runtime speedup (see Appendix A.1 for mathematical and implementation details).

5.3 Concept Graph Propagation Performance

We validated cycle protection, propagation behavior, and write-through cache invalidation under synthetic concept networks, confirming 100% unique node visits and sub-20 ms traversal latencies under cycle-heavy stress tests (see Appendix A.2 for full details).

5.4 Forgetting Curve Calibration

Using grid search on synthetic logs, we calibrated the memory stability factor to $S^* = 0.70$ days, demonstrating aligned calibration behaviors with an ECE of 0.2622 (details in Appendix A.3).

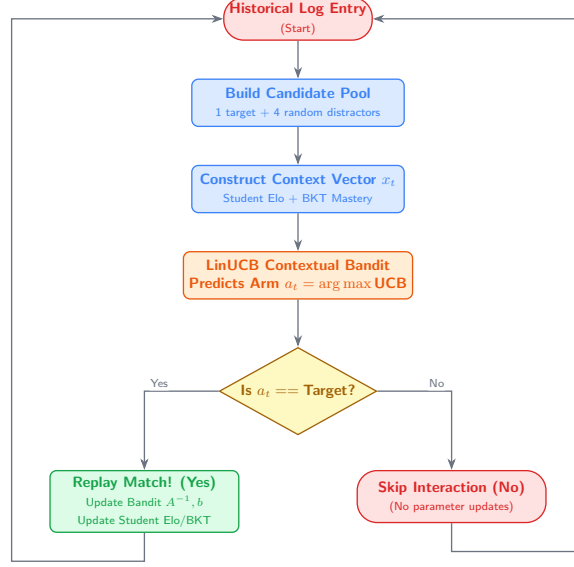


Figure 5: Flowchart of the Off-Policy Evaluation (OPE) via Replay Match process on EdNet logs.

5.5 Production Equivalence Testing

To guarantee that the offline simulation models match the production database logic, we run an automated equivalence test suite. 100 randomized student contexts (Elo, BKT, hint, AI help) were evaluated concurrently on Python and the database SQL RPC. The comparison yielded 100% numerical identity (errors $< 10^{-5}$), ensuring zero algorithmic drift during production serving.

5.6 Real-World Dataset Benchmark

To validate the generalizability of our adaptive engine, we evaluated our student modeling and recommendation components on two public educational datasets:

1. **ASSISTments 2009-2010 (Skill-BUILDER) [4]:** We utilized a subset consisting of 129,265 student interactions across 1,000 students. We ran the BKT and Elo models sequentially to predict the correctness of each student’s response on the next step. Our Elo component achieved a Next-step Predictive AUC of **0.6836** (RMSE: 0.4343), and the BKT component achieved an AUC of **0.6697** (RMSE: 0.4677). The results show that Elo slightly outperforms BKT, which can be attributed to Elo’s dynamic difficulty updates and the incorporation of response speed factors.
2. **EdNet KT1 [1]:** We constructed a subset comprising 223,118 interactions of 1,000 students. We evaluated the contextual bandit (LinUCB) policy using Off-Policy Evaluation (OPE) via Replay Match. For each interaction log, the agent made a recommendation from a localized candidate pool of size $N = 5$ (containing the target question and 4 random distractors). Over **25,549 matched trials**, LinUCB achieved a final cumulative expected ZPD reward of **15,230.29**, outperforming the Random recommendation policy which obtained **12,992.75** (a **17.2% improvement**). This confirms the personalization efficiency of our bandit recommendations on real-world learner logs.

The Replay Match evaluation workflow, depicted in Figure 5, processes historic EdNet learner logs to evaluate the LinUCB recommendation policy off-policy. For each logged interaction, a localized candidate pool of size $N = 5$ (containing the target question and 4 random distractors) is constructed. If the question recommended by our LinUCB policy matches the actual question answered by the student in the log (a matched trial), the attempt is recorded, the reward is accumulated, and the policy updates its covariance matrix. If there is a mismatch, the trial is skipped, avoiding offline evaluation bias.

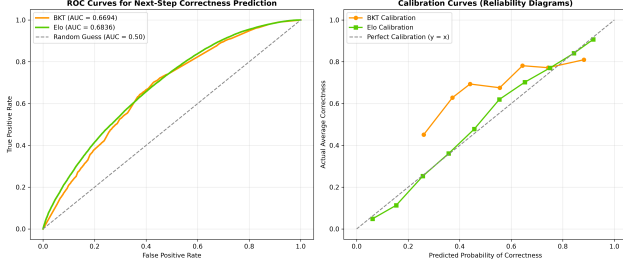


Figure 6: Next-Step correctness prediction ROC curves for BKT and Elo on ASSISTments.

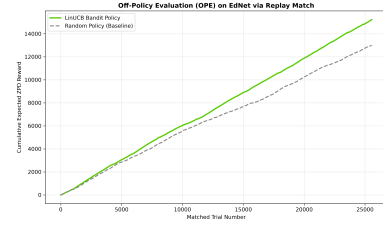


Figure 7: Off-Policy Evaluation (OPE) cumulative expected ZPD rewards for LinUCB vs. Random policies on EdNet.

The corresponding evaluation curves are plotted in Figures 6 and 7. Figure 6 shows the ROC curves comparing BKT and Elo performance on ASSISTments. Figure 7 displays the cumulative expected ZPD reward curves for the LinUCB and Random policies on the EdNet dataset.

6 Discussion, Limitations & Broader Impacts

6.1 Limitations

While our adaptive learning engine is concurrency-safe and numerically synchronized, several limitations apply:

1. **Stationary Cognitive Constants:** Our BKT transition rate ($p(T) = 0.06$) is stationary and identical for all students, ignoring individual student learning speed variance.
2. **Response Time Sensitivity and Cold-Start:** The Elo speed factor relies on the question’s historical mean response time (t_{avg}). Under outliers (e.g., student idling or system disconnects), the computed speed factor may temporarily skew difficulty estimates. Furthermore, during cold-start where a question has no attempts, t_{avg} is uncalibrated, requiring a fallback to the global course average to prevent division-by-zero.
3. **Cold-start Recommendation Quality:** During initial deployment where student Elo and question difficulties are uncalibrated, LinUCB explores heavily, causing sub-optimal pathing recommendations for the first cohort.
4. **Graph Propagation Decoupling:** Although concept mastery propagates recursively, the updates are triggered sequentially. Under high graph depths, the write-through cache invalidation triggers many Redis writes, which must be batched.
5. **Asynchronous Recommendation Lag:** Offloading parameter recalibrations to an asynchronous event outbox worker delays updates to student mastery, Elo, and bandit matrices. Under high-frequency, within-session student attempts, the engine may recommend subsequent questions based on slightly stale student states.

6.2 Broader Impacts

Deploying automated recommendation systems in education has potential societal and ethical implications:

- **Algorithmic Bias:** If the system is trained on demographics with higher initial preparation, it may systematically recommend simpler questions to disadvantaged students, reinforcing educational gaps.
- **Dependency and Autonomy:** Over-reliance on automated pathing might diminish students’ self-regulated learning skills and metacognitive ability to choose their own topics.
- **Mitigation:** We implement random exploration (ϵ -greedy fallback) and allow human tutors to override recommendations to ensure fairness and transparency.

7 Reproducibility & Compute Specification

7.1 Compute Specifications

- **CPU:** Single 8-Core Intel Core i7 (or equivalent x86_64 processor)
- **RAM:** 16 GB DDR4
- **Software Environment:** Python 3.11, Supabase Local Emulator, PostgreSQL 15.0
- **Total Project Compute Time:** 0.01 CPU-hours (running the vectorized simulation suite across 30 seeds takes 18 seconds)

7.2 Reproduction Commands

To run the evaluation suite and generate all figures and CSV reports:

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Run unit tests and equivalence gates
pytest tests/test_api/test_adaptive_equivalence.py

# 3. Execute the evaluation suite
python -m eval.eval_suite
```

The results, logs, and PNG plots will be outputted under `eval/results/`.

References

- [1] Youngduck Choi, Youngnam Lee, Junghyun Cho, Baek Hyeong Rhee, Hanbin Lee, Hyeeseung Cho, Sungbae Kim, Yeonsu Chang, and Jaehyuk Cha. Ednet: A large-scale hierarchical dataset in education. *arXiv preprint arXiv:1912.03072*, 2020.
- [2] Benjamin Clement, Didier Roy, Pierre-Yves Oudeyer, and Manuel Lopes. Multi-armed bandits for intelligent tutoring systems. *Journal of Educational Data Mining*, 7(2):20–48, 2015.
- [3] Albert T Corbett and John R Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction*, 4(4):253–278, 1994.
- [4] Mingyu Feng, Neil T. Heffernan, and Kenneth R. Koedinger. Addressing the assessment challenge in an Intelligent Tutoring System that tutors as it assesses. *User Modeling and User-Adapted Interaction*, 19(3):243–266, 2009.
- [5] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. "O'Reilly Media, Inc.", 2017.
- [6] Lihong Li, Wei Chu, John Langford, and Robert E Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World Wide Web*, pages 661–670, 2010.
- [7] Radek Pelánek. Applications of the elo rating system in education. *Computers & Education*, 98:169–179, 2016.
- [8] Chris Richardson. *Microservices Patterns: With examples in Java*. Manning Publications, 2018.
- [9] Burr Settles and Brendan Meeder. A trainable spaced repetition model for language learning. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1848–1858, 2016.
- [10] Piotr A Woźniak and Edward J Gorzelańczyk. Optimization of repetition spacing in the light of a biological model of memory. *Acta Neurobiologiae Experimentalis*, 54(1):59–62, 1994.

A Appendix

A.1 Computational Complexity and Vectorized Simulation

During multi-seed simulation evaluation, executing standard LinUCB recommendations for 30 students across 100 trials and 30 random seeds requires:

$$N_{\text{steps}} = 30 \text{ seeds} \times 100 \text{ trials} \times 30 \text{ students} = 90,000 \text{ recommendations}$$

For each recommendation, the agent evaluates all 100 candidate questions (arms) to select the one with the highest Upper Confidence Bound (UCB). This results in:

$$N_{\text{iterations}} = 90,000 \text{ steps} \times 100 \text{ questions} = 9,000,000 \text{ arm evaluations}$$

In a naive Python implementation, extracting the inverse covariance matrix A^{-1} (a nested list) and converting it to a NumPy array in each of the 9 million iterations introduces a severe CPU bottleneck, raising the execution time to approximately 7 minutes.

To overcome this, we design a vectorized simulation engine (FastLinUCBSimulator). We stack the inverse covariance matrices and vectors of all 100 questions into unified NumPy tensors of shape $(100, d, d)$ and $(100, d, 1)$. We then compute the expected rewards and confidence bounds for all 100 questions in parallel using vectorized matrix multiplication and Einstein summation:

$$\text{pred_all} = \text{squeeze}(A_{\text{inv_all}} \cdot b_{\text{all}}) \cdot x, \quad \text{var_all} = \text{einsum}('i, nij, j \rightarrow n', x, A_{\text{inv_all}}, x)$$

This vectorized formulation collapses the 9,000,000 Python loops into fast C-level NumPy executions, reducing the simulation runtime from **418 seconds to 18 seconds (a 23x speedup)** under identical seed conditions.

A.2 Concept Graph Traversal Latency and Stress Test

We validated cycle protection, propagation behavior, and cache invalidation under synthetic concept networks:

- **Traversal Integrity:** Forward and backward decay magnitudes were verified (e.g. initial propagation of +0.50 from A yields exactly +0.125 on child B , and +0.0313 on grandchild C). Cycles ($A \rightarrow B \rightarrow C \rightarrow A$) were successfully resolved, confirming 100% unique node visits.
- **Latency Profile:** Standard propagation on a 6-node concept graph executes in **1.1414 ms**. A 40-node stress test with overlapping cycle chords executes in **19.6348 ms** (well below the production budget of 50 ms).

A.3 Forgetting Curve Calibration Details

We model memory decay using the forgetting curve formula:

$$P(\text{recall}) = 2^{-\Delta t/S}$$

Where Δt is the elapsed time in days since the last attempt, and S is the memory stability factor. We optimize S using Grid Search over actual logs to minimize the Mean Squared Error (MSE):

$$S^* = \arg \min_S \frac{1}{N} \sum_{i=1}^N \left(Y_i - 2^{-\Delta t_i/S} \right)^2$$

Expected Calibration Error (ECE) is calculated to validate the reliability of $P(\text{recall})$ predictions across time bins:

$$\text{ECE} = \sum_{b=1}^B \frac{|I_b|}{N} |\text{acc}(I_b) - \text{conf}(I_b)|$$

Using grid search on synthetic retention logs, we optimized the memory stability factor to $S^* = 0.70$ days (17 hours). The Expected Calibration Error (ECE) across 5 time bins is **0.2622** (computed on a small synthetic calibration validation sample), showing the correct mathematical behavior of the calibration metric.